QaAgent-Pro

Zybra CRM Personal QA Automation Agent

PRD + Technical Design + Test Design + Research Refresh v2

Prepared for: Zybra CRM QA/Product/Engineering

Date: 19 June 2026

Refresh v2 correction

This version explicitly adds the refresh/persistence suite, full CRM blueprint coverage, UX/function/feature testing, next-build recommendations, and a redesigned layout that avoids dense broken tables.

- Standalone repo: QaAgent-Pro
- Excel-only final agent report with embedded screenshots
- Codex/no-API primary with optional Groq/API mode
- Archive allowed in staging; delete included only in destructive full-regression mode

1. Document Control

Item	Decision
Document version	v2 - research refresh and layout repair
Project name	QaAgent-Pro
Target product	Zybra CRM
Repository strategy	New standalone repository. Reuse proven concepts from BAKUGOS1/QaAgent; do not simply fork and rename.
Primary output	Excel workbook only for user-facing QA output; PDF is this planning artifact.
Tone	CTO-grade architecture plus practical QA-lead execution plan.
Key correction from v1	Add refresh/persistence testing and explicit next-build improvement reporting.

1.1 Locked Product Decisions

#	Decision	Locked value
1	Agent name	QaAgent-Pro
2	Agent report format	Excel only, with embedded screenshots
3	Archive testing policy	Archive/unarchive allowed on staging
4	Delete testing policy	Delete included in destructive full-regression suite
5	Agent intelligence mode	Codex/no-API plus optional Groq/API
6	Document style	Mixed CTO-grade and QA-lead practical blueprint
7	Repo approach	Completely new repository

Important safety interpretation

Because archive and delete are destructive operations, the design allows them only after staging-environment verification, tenant allowlist confirmation, and explicit destructive-run configuration. This keeps the user decision intact while preventing accidental production damage.

2. Audit of Previous PDF and What Was Missing

The previous PDF had the right direction, but it was not complete enough for the real Zybra CRM assignment. It also used dense tables and tight layout, which caused readability and break concerns. This v2 document fixes both content and layout.

Gap	Impact	v2 Fix
Refresh was missing	CRM data can pass a toast check but disappear after refresh/reload.	Added a dedicated Refresh and Persistence Suite across create, edit, filter, archive, unarchive, delete, pipeline, and manage-column flows.
Blueprint conformance not explicit	Agent could test generic CRM flows instead of the supplied Excalidraw blueprint.	Added CRM Blueprint Conformance Mode and module-by-module blueprint mapping.
UX/function/feature distinction was weak	Bugs, UX issues, feature gaps, and next-build ideas would mix together.	Added separate result categories: Functional Bug, UX Issue, Feature Gap, Data Integrity Issue, Performance Issue, Release Blocker, Next-Build Recommendation.
Next-build recommendations missing	Agent would only report problems, not define what to improve in next release.	Added Product Improvement Engine and Next Build Backlog sheet in Excel output.
Standalone repo decision unclear	Previous plan leaned on existing repo/fork strategy.	Locked as new repo while reusing architecture ideas from QaAgent.
Safety too generic	Archive/delete choices were not fully reflected.	Added staging-only destructive policy with gates and audit evidence.
Tables too dense	PDF looked broken and hard to read.	This document uses smaller tables, fewer columns, more pages, and safer page breaks.

2.1 v2 Objective

QaAgent-Pro is not just an automated bug finder. It is a CRM quality intelligence agent that verifies the supplied CRM blueprint, tests the UX and functional workflows, detects gaps, and produces build-ready recommendations for the next Zybra CRM release.

3. Research Findings

The design combines current Playwright capabilities, the existing QaAgent repository direction, and browser-agent research. The conclusion is clear: use deterministic CRM playbooks as the foundation, then add controlled agent intelligence for planning, selector healing, evidence collection, and report synthesis.

3.1 Research-backed Design Rules

Research point	Finding	Design consequence
Playwright auth/session	Playwright supports saving authenticated browser state and warns that auth files can contain sensitive cookies/headers.	QaAgent-Pro stores auth under .auth and never commits it.
Playwright best practice	Tests should verify user-visible behavior and remain isolated.	QaAgent-Pro validates rendered UI plus final persisted data state, not only APIs or toasts.
Playwright codegen	Playwright can record flows and generate resilient locators prioritizing role, text, and test-id locators.	Initial CRM selectors can be captured quickly, then converted to stable page objects.
Page Object Model	Playwright recommends page objects for large maintainable suites.	Each CRM module gets dedicated page objects and playbooks.
Trace/screenshots/network	Playwright supports traces, screenshots, videos, console events, and network inspection.	Every bug gets evidence; failed scenarios carry screenshots, trace, console errors, and failed network calls.
Accessibility testing	Automated accessibility checks catch many issues but cannot replace manual assessment.	QaAgent-Pro includes automated accessibility scans plus manual UX heuristics.
Web-agent reliability research	Recent browser-agent studies report real-world reliability and action-failure problems.	The agent does not randomly click. It follows CRM-specific deterministic playbooks.

3.2 Existing QaAgent Repo Findings

The BAKUGOS1/QaAgent repository is useful as a reference implementation. It already focuses on local-first TypeScript + Playwright QA automation, evidence capture, selector healing, safety rules, console/network checks, and Excel-first reports. QaAgent-Pro should reuse these concepts, not the exact repo structure blindly.

Reusable concept from QaAgent	How QaAgent-Pro will use it
Local-first browser engine	Run CRM tests from developer machine or QA laptop without required cloud dependency.
Codex/no-API mode	Use Codex/chat reasoning for planning while Playwright performs local browser actions.
Optional Groq/API mode	Enable autonomous report summarization and natural-language task execution later.
Safety guard	Adapt to staging-only archive/delete policies.
Excel-first reporting	Keep Excel as the final operational artifact for QA/dev handoff.
Selector healing	Use controlled healing when locators change, but never hide a real UI regression.
Command log and evidence	Attach reproducible evidence to every defect.

3.3 Agent Strategy Conclusion

Core recommendation

QaAgent-Pro should be a hybrid deterministic QA agent: fixed CRM playbooks first, browser-state intelligence second, optional LLM autonomy third. This gives reliable QA results while still making the agent useful for UX and next-build recommendations.

4. Uploaded Excalidraw CRM Blueprint Audit

The uploaded Excalidraw file contains 5557 diagram elements and 2100 text labels. It covers the CRM navigation, Leads, Deals, Activities, Products, archive/unarchive, pipeline, filters, notes, communication actions, and subscription/product plans.

Blueprint area	Detected from file	Agent interpretation
Main navigation	Dashboard, Leads, Deals, Activities, Campaigns, Referrals, Tasks, Reports, Products, Settings	Every sidebar item becomes a navigation smoke test and visibility check.
Lead table	Inbox, Archive, Filter, Search, Import, Export, Manage Columns	Core table workflow and data-grid regression suite.
Lead details	Details, labels, owner, value, expected close date, source channel, history, notes	Detail drawer/page conformance and edit validation.
Lead activity	Call, Email, Whatsapp, Note, Activity, outcomes, due states	Communication and activity scheduling suite.
Conversion	Convert to Deal, pipeline, stage, value, product, close date, probability	Cross-module Lead-to-Deal workflow.
Deals	Pipeline, stages, Add Deal, Mark Won, Mark Lost, Lost Reason, Archive	Pipeline lifecycle and drag/drop/persistence suite.
Products	Add or Edit Products, Subscription, Growth plans, billing frequency, tax	Product and subscription plan test suite.
Refresh	Not present in blueprint	Added in v2 as a required QA suite because persistence and state recovery are essential.

4.1 Blueprint Rule

The blueprint becomes the first expected-product oracle. If a control or flow exists in the Excalidraw design but is missing from the app, the agent should report it as a Product Conformance Gap. If the app has extra behavior not in the blueprint, the agent should record it as Needs Product Confirmation, not automatically as a bug.

4.2 Blueprint Coverage Modes

Mode	Purpose	Example result
Blueprint Conformance	Check whether the CRM app matches the supplied screens and controls.	Missing Manage Columns button in Leads table.
Functional Regression	Check whether existing workflows behave correctly.	Lead saves but does not appear after search.
UX Review	Check clarity, feedback, layout, discoverability, empty/error states.	Archive confirmation text unclear or button hidden.
Feature Gap Discovery	Find missing capabilities implied by CRM flow.	No refresh button or no persistence after reload.
Next Build Recommendation	Convert confirmed gaps into build-ready backlog items.	Add table refresh action after bulk archive.

5. Product Vision

QaAgent-Pro will be an internal CRM quality automation platform. It will test Zybra CRM the way a QA lead would: verify the blueprint, run happy-path and negative-path flows, capture evidence, retest fixes, and recommend what the next build should improve.

5.1 Product Goals

- Automate repetitive CRM regression testing from a single command.
- Validate the supplied CRM blueprint, not generic CRM assumptions.
- Detect functional, UX, feature, data, performance, and release-blocking issues.
- Generate an Excel-only QA workbook with embedded screenshots and developer-ready evidence.
- Define the next build backlog from confirmed defects and product gaps.
- Support safe destructive testing in staging only.
- Support both local deterministic execution and optional AI-assisted summaries.

5.2 Non-Goals

- Do not run random unrestricted autonomous browsing against production.
- Do not depend on source-code access for black-box CRM testing.
- Do not send real emails, WhatsApp messages, calls, invites, or payments.
- Do not delete production data or export real customer-sensitive data.
- Do not treat toast messages as proof of success without persistence verification.

5.3 Success Definition

Success definition

QaAgent-Pro is successful when Zybra can run a CRM regression suite before every release and receive a clean Excel workbook showing bugs, UX issues, feature gaps, refresh/persistence failures, and next-build recommendations with screenshots.

6. Users and Operating Context

User	Need	QaAgent-Pro response
QA tester	Run repeatable CRM tests quickly.	CLI commands and Playwright headed mode for debug.
Developer	Understand and reproduce bugs.	Steps, expected/actual, screenshot, console/network data.
Product manager	Know what is missing versus CRM blueprint.	Feature Gap and Next Build Backlog sheets.
CTO/lead engineer	Assess release risk.	Summary dashboard with blockers, high-risk flows, and pass/fail count.
Support/implementation team	Verify customer-facing workflows.	Workflow-oriented reports grouped by module.

7. Agent Operating Modes

QaAgent-Pro should support multiple operating modes because CRM testing is not one task. Each mode has different depth, safety, and output expectations.

Mode	Command idea	Purpose	Safety level
Blueprint audit	npm run qa:blueprint	Check app screens and controls against Excalidraw-derived blueprint.	Read-only except navigation.
Smoke	npm run qa:smoke	Quick login, sidebar, major pages, table load.	Safe.
Leads MVP	npm run qa:leads	Full Leads workflow from create to archive/unarchive and conversion.	Creates and edits test data.
Functional regression	npm run qa:regression	Run all non-destructive workflows.	Creates/edits/archives staging data.
UX audit	npm run qa:ux	Check discoverability, empty states, accessibility, layout, labels, friction.	Mostly read-only.
Refresh suite	npm run qa:refresh	Check refresh/reload persistence and stale UI states.	Uses test data and stage changes.
Destructive full regression	npm run qa:full-destructive	Includes delete, destructive settings-like flows, bulk archive/delete where allowed.	Requires explicit env flag and staging gate.
Retest	npm run qa:retest --bug BUG-ID	Re-run fixed issues and update status.	Only scoped to previous bug.

7.1 Result Categories

Category	Meaning	Example
Functional Bug	Expected workflow fails.	Lead created but not searchable.
UX Issue	Workflow works but is confusing or difficult.	Archive action hidden behind unlabeled icon.
Feature Gap	Expected control/behavior from blueprint is missing.	No Manage Columns option in Leads table.
Data Integrity Issue	UI/API/data persistence mismatch.	Toast says saved but reload loses record.
Performance Issue	Threshold exceeded.	Search takes more than 2 seconds.
Accessibility Issue	Keyboard, contrast, label, focus, or screen-reader problem.	Email input has no accessible label.
Release Blocker	High-risk issue that should block build release.	Delete removes wrong record.
Next-Build Recommendation	Improvement that should become product backlog.	Add refresh button and stale-state indicator.

8. System Architecture

CLI Command
-> Config and Safety Gate
-> CRM Blueprint Loader
-> Session/Login Manager
-> Planner
-> Playbook Runner
-> Page Objects
-> Playwright Browser Engine
-> Validators and Oracles
-> Evidence Collector
-> Bug/UX/Feature-Gap Classifier

-> Excel Reporter

-> Next-Build Backlog Generator

8.1 Component Responsibilities

Component	Responsibility
CLI	Select mode, module, headed/headless, safety profile, output path.
Config loader	Validate env vars, staging URL, credentials, destructive flags.
Blueprint loader	Load CRM expected controls, flows, modules, validation rules, and test IDs.
Session manager	Login once, store auth state locally, refresh expired sessions.
Planner	Select playbooks based on mode and module.
Playbook runner	Execute ordered scenarios and preserve evidence after every significant action.
Page objects	Maintain stable module-specific browser operations.
Validators	Check UI, API, persistence, refresh, accessibility, and performance.
Evidence collector	Save screenshots, trace, console logs, network errors, DOM state, timings.
Reporter	Generate Excel workbook with clean report sheets and embedded screenshots.
Backlog generator	Convert issues/gaps into prioritized next-build items.

9. Repository and Package Design

Because the final choice is a new standalone repository, QaAgent-Pro should be cleanly structured around Zybra CRM rather than being a generic website crawler with CRM tests added later.

9.1 Suggested Repository

```
qaagent-pro/
  apps/
    cli/
  packages/
    browser-engine/
    crm-blueprint/
    crm-playbooks/
    evidence/
    reporters/
    safety/
    validators/
  tests/
    smoke/
    leads/
    deals/
    activities/
    products/
    refresh/
    destructive/
  artifacts/
    reports/
    screenshots/
    traces/
    state/
  docs/
    prd-tdd/
    runbooks/
    crm-blueprint/
  .env.example
  package.json
  playwright.config.ts
  README.md
```

9.2 Key Files

File	Purpose
docs/crm-blueprint/zybra-crm-blueprint.yaml	Source-of-truth expected CRM controls, flows, labels, and validation rules.
packages/crm-playbooks/src/leads.playbook.ts	Leads workflow automation.
packages/crm-playbooks/src/deals.playbook.ts	Deals pipeline and lifecycle tests.
packages/validators/src/refresh.validator.ts	Refresh/reload persistence checks.
packages/safety/src/destructive-gate.ts	Staging and destructive action guard.
packages/reporters/src/excel.reporter.ts	Excel-only user-facing report.
tests/refresh/*.spec.ts	Dedicated persistence and reload test cases.

9.3 Environment Contract

```
CRM_BASE_URL=https://staging-crm.example.com
CRM_EMAIL=qa@example.com
CRM_PASSWORD=********
CRM_TENANT=qa-staging
CRM_ENVIRONMENT=staging
HEADLESS=false
QA_AGENT_PREFIX=QA_AGENT_
ALLOW_ARCHIVE=true
ALLOW_DELETE=false
ALLOW_DESTRUCTIVE=false
REPORT_FORMAT=xlsx
AI_MODE=codex
GROQ_API_KEY=
GROQ_MODEL=
```

Destructive mode rule

Delete tests require ALLOW_DELETE=true and ALLOW_DESTRUCTIVE=true. The agent must refuse delete tests if CRM_ENVIRONMENT is not staging or if the URL is not in the staging allowlist.

10. CRM Blueprint Configuration

The CRM blueprint should be converted into a structured config. This allows the agent to test the current app against the expected product design and also helps future builds update the test plan without rewriting all code.

10.1 Example Blueprint YAML

```
modules:
  leads:
    navigation_label: Leads
    required_controls:
      - Inbox
      - Archive
      - Filter
      - Add Lead
      - Import Data
      - Export Data
      - Manage Columns
      - Search
    detail_controls:
      - Details
      - Labels
      - Owner
      - Value
      - Expected Close Date
      - Source Channel
      - History
      - Note
      - Activity
      - Call
      - Email
      - Whatsapp
      - Convert to Deal
    business_rules:
      - archived_lead_is_read_only
      - unarchive_required_before_edit
      - converted_lead_creates_deal
      - table_refresh_or_reload_must_preserve_saved_data
```

10.2 Blueprint Versioning

Field	Use
blueprint_version	Keeps test expectations tied to a design revision.
source_file	References Excalidraw/PDF/source notes.
module_status	Marks flows as MVP, next build, future, or deprecated.
owner	Product/QA owner for deciding ambiguous behavior.
last_reviewed_at	Prevents stale assumptions.
confidence	High when explicitly in blueprint; Medium when inferred; Low when needs confirmation.

11. Refresh and Persistence Requirement

This is the main new requirement added in v2. Refresh is not visible in the Excalidraw file, but it must be added because CRM workflows often show temporary success while actual persistence fails. QaAgent-Pro must verify state after UI refresh and browser reload.

11.1 Refresh Types

Type	What agent does	Why it matters
Soft table refresh	Click refresh button if present or re-open module/list.	Checks whether latest server state appears without browser reload.
Search refresh	Clear and re-run search after save/archive/delete.	Avoids stale filtered results.
Browser reload	Use page.reload() after create/edit/archive/delete.	Confirms persistence across actual reload.
Navigation refresh	Navigate away and back to module.	Detects state that only exists in current component memory.
Session refresh	Reopen browser with saved auth and check data.	Confirms data survives new session.
API verification	Optionally verify via network/API response if safe and available.	Catches toast/API mismatch.

11.2 Refresh Test Cases

ID	Area	Agent action	Expected result
REF-001	Leads create	Create lead, reload page, search lead.	Lead remains visible with same values.
REF-002	Leads edit	Edit owner/value/label, reload detail.	Updated values persist.
REF-003	Lead archive	Archive lead, reload Inbox and Archive.	Lead absent from Inbox and present in Archive.
REF-004	Lead unarchive	Unarchive lead, reload Archive and Inbox.	Lead returns to Inbox and leaves Archive.
REF-005	Lead convert	Convert lead to deal, reload Deals.	New deal exists in expected pipeline/stage.
REF-006	Filters	Apply filter, refresh list.	Filter state is either preserved or intentionally reset; behavior must be consistent and documented.
REF-007	Manage columns	Hide/show column, reload page.	Column preference follows product decision.
REF-008	Deals stage	Move deal stage, reload board.	Card remains in new stage.
REF-009	Won/Lost	Mark deal won/lost, reload board/detail.	Status and reason persist.
REF-010	Delete full regression	Delete QA record, reload table/search.	Deleted record does not reappear.

11.3 Refresh Bug Classification

Symptom	Classification	Priority guidance
Toast says saved but data disappears after reload.	Data Integrity Issue	High/Critical depending on module.
Table remains stale until manual reload.	UX Issue or Functional Bug	Medium/High.
Filter changes after refresh without explanation.	UX Issue	Medium.
Archive/delete appears successful but record remains active.	Functional Bug	High.
Converted deal not visible after reload.	Cross-module Data Issue	Critical.

12. Safety, Privacy, and Destructive-Action Policy

The user selected archive allowed and delete included in full regression. The agent must still protect production and real customer data. Safety is implemented as a runtime gate, not a suggestion.

12.1 Safety Profiles

Profile	Allowed	Blocked
safe	Navigation, screenshots, search, filter, view details, create QA data.	Archive, delete, settings, exports, real messaging.
staging-write	Create/edit QA data, archive/unarchive, convert to deal, product test data.	Delete, billing, real messaging, sensitive export.
destructive-staging	Delete QA records, stage delete, destructive bulk flows.	Production URL, real customer records, payments, real messages.
read-only-audit	Blueprint conformance, UX review, accessibility scans.	All data modifications.

12.2 Mandatory Gates

- CRM_ENVIRONMENT must equal staging for archive/delete tests.
- CRM_BASE_URL must match allowlisted staging domains.
- Agent must create or identify test records before destructive actions whenever possible.
- Destructive run must write an audit log before and after each destructive operation.
- If a record does not have QA_AGENT_ prefix, the agent must pause or require explicit allowlist.
- Export tests must avoid sensitive customer data and should prefer mock/test tenant data.
- Real Email, WhatsApp, Call, payment, subscription billing, and user invitation actions remain blocked unless separately authorized.

12.3 What Delete Testing Means

Delete testing is included in the full regression suite, but it is not part of the first MVP. Delete tests must verify expected preconditions, confirmation dialogs, permission errors, final absence from the table/search, and reload persistence. If the product rule says only archived records can be deleted, the agent must test both invalid direct-delete and valid archive-then-delete paths.

13. Excel-Only Reporting Specification

The agent output is Excel only. Markdown/JSON may exist internally for debugging, but the final deliverable to QA/Product/Dev is a clean Excel workbook with embedded screenshots.

13.1 Workbook Sheets

Sheet	Audience	Purpose
Bug Report	QA, developers	Clean list of functional bugs and release blockers.
UX Issues	Product, design, QA	Discoverability, friction, accessibility, layout, copy, empty/error-state issues.
Feature Gaps	Product, engineering	Missing controls/flows compared to blueprint.
Next Build Backlog	PM, engineering lead	Prioritized improvement tasks derived from findings.
Refresh Persistence	QA, backend/frontend dev	Reload and stale-state validation results.
Summary	Leadership and team leads	Counts by module, severity, category, release risk.
Evidence Log	Developers	Screenshots, console errors, network failures, trace paths.
Run Metadata	QA owner	Config, environment, browser, commit/build if available, run timestamps.

13.2 Bug Report Columns

Column	Meaning
Module	CRM area: Leads Table, Lead Detail, Deals Pipeline, Products, etc.
Issue	Short defect title.
Description	What happened, where, and why it is wrong.
Steps to Reproduce	Numbered steps.
Expected Result	Product/business expectation.
Actual Result	Observed app behavior.
Priority	Critical, High, Medium, Low.
Status	Open, Needs Verification, Fixed, Retest Failed, Blocked.
Screenshot	Embedded image in the row.
Evidence Ref	Trace/log/raw artifact reference.

13.3 Next Build Backlog Columns

Column	Meaning
Backlog ID	Example: QAP-NB-001.
Type	Bug Fix, UX Improvement, Feature Gap, Reliability, Testability.
Recommendation	Clear action for next build.
Reason	Evidence-based explanation.
Affected Modules	One or many CRM modules.
Priority	P0/P1/P2/P3.
Acceptance Criteria	How the team knows it is fixed.
Linked Evidence	Bug ID, screenshot, trace.

Report rule

Excel must be readable without manual resizing: wrapped text, frozen headers, practical column widths, embedded screenshots, grouped technical evidence, and clean first sheets for non-technical stakeholders.

14. Test Design Overview

The test design follows risk-based CRM QA. High-risk workflows are create/save/update, data loss, cross-module conversion, archive/delete, import/export, pipeline state changes, and communication actions.

14.1 Test Profiles

Profile	Depth	Primary purpose
Smoke	Low	Check login, sidebar, module pages, critical buttons.
Functional	Medium	Check expected workflows and validations.
UI/UX	Medium	Check usability, labels, focus, loading, empty/error states.
Regression	High	Run module workflows end-to-end.
Refresh	High	Verify persisted state after reload/navigation/session reuse.
Destructive	Very high	Archive/delete/bulk destructive flows on staging only.
Accessibility	Medium	Automated checks plus manual heuristics.
Performance	Medium	Page load, search, save, archive, pipeline action timings.

14.2 Global CRM Tests

ID	Area	Agent action	Expected result
CRM-001	Login	Open CRM and login with test credentials.	User lands on dashboard or configured start page.
CRM-002	Sidebar	Verify all blueprint modules.	Dashboard, Leads, Deals, Activities, Campaigns, Referrals, Tasks, Reports, Products, Settings are visible or intentionally hidden by role.
CRM-003	Navigation	Open each sidebar item.	Page loads without console/network critical errors.
CRM-004	Role/permission	Attempt restricted actions if role data available.	Correct blocked state or permission message.
CRM-005	Responsive basics	Run selected smoke tests on desktop and common laptop viewport.	No unusable layout or hidden primary action.
CRM-006	Error states	Simulate empty searches and invalid forms.	Helpful feedback appears.

14.3 Oracle Model

Oracle	Checks
UI oracle	Visible text, controls, modal/drawer open state, button state, icons, table rows.
Data oracle	Record exists or not after search/reload/navigation.
Network oracle	Failed HTTP request, API error payload, toast/API mismatch.
Console oracle	Unhandled JS error, uncaught exception, failed resource.
Performance oracle	Duration thresholds for load/search/save/archive/delete.
Blueprint oracle	Expected screen/control/flow exists as per Excalidraw config.
UX oracle	Discoverability, clarity, focus order, loading feedback, empty/error state.

15. Leads Module Test Design

Leads is the MVP module because it has the densest CRM flow in the blueprint: table, filters, detail, note, activity, call/email/WhatsApp, conversion, archive/unarchive, import/export/manage columns, and bulk actions.

15.1 Leads MVP Scope

- Open Leads page and verify Inbox/Archive.
- Verify Add Lead, Filter, Search, Import, Export, Manage Columns.
- Create valid lead with unique QA_AGENT_ data.
- Validate required fields, invalid email, invalid mobile, duplicate behavior.
- Search created lead and verify table data.
- Open lead detail and edit focus fields.
- Add note and schedule activity.
- Test communication action visibility and non-send safety behavior.
- Convert lead to deal and verify deal appears.
- Archive/unarchive and verify after reload.
- Run refresh/persistence checks after create/edit/archive/unarchive/convert.

15.2 Leads Test Cases

ID	Scenario	Agent action	Expected result
LEAD-001	Page load	Open Leads from sidebar.	Leads table appears under threshold.
LEAD-002	Blueprint controls	Verify Inbox, Archive, Filter, Add Lead, Import, Export, Manage Columns, Search.	All expected controls visible or role exception logged.
LEAD-003	Required validation	Open Add Lead and submit empty form.	Required errors appear; no lead created.
LEAD-004	Invalid mobile	Enter invalid mobile and save.	Validation error appears.
LEAD-005	Invalid email	Enter invalid email and save.	Validation error appears.
LEAD-006	Create valid lead	Fill contact, business, mobile, email, address, label, owner, value, source.	Lead saved and visible.
LEAD-007	Duplicate check	Create same email/mobile again.	Duplicate warning or expected rule behavior occurs.
LEAD-008	Search	Search created lead by name/mobile/email.	Correct row appears.
LEAD-009	Detail open	Open lead detail/drawer.	Details, focus, history, actions visible.
LEAD-010	Edit focus fields	Edit label/owner/value/expected close/source.	Values save and persist after reload.
LEAD-011	Note	Add note.	Note appears in history/timeline.
LEAD-012	Activity	Schedule activity.	Activity appears with expected date/status.
LEAD-013	Call action	Click Call with safety mode.	Call flow/instruction opens; no real call sent.
LEAD-014	Email action	Open email composer.	From/To/Subject/Template/Send controls visible; real send blocked unless allowed.
LEAD-015	WhatsApp action	Click WhatsApp.	Expected flow opens; real message blocked unless allowed.
LEAD-016	Convert to deal	Convert lead with pipeline/stage/value/product/close date.	Deal created in expected pipeline/stage.
LEAD-017	Archive	Archive lead.	Lead moves to Archive and is read-only.
LEAD-018	Unarchive	Unarchive lead.	Lead returns to Inbox and can be edited.
LEAD-019	Bulk archive	Select multiple staging leads and archive.	Confirmation appears and selected leads move to Archive.
LEAD-020	Refresh persistence	Reload after every state change.	Data remains consistent after reload.

15.3 Lead Filter Coverage

Filter	Operators/Values	Expected check
Owner	is, is not; sample owners from blueprint	Rows match selected owner.
Labels	Hot, Warm, Cold; includes/excludes	Rows match label filter.
City	Ahmedabad, Surat, Rajkot examples	Rows match city.
Activity Date	is, is not, before, after, between	Rows match date range.
Source Channel	Cold Calling, Website, Social Media, Event, Referral, Other	Rows match source.
No activity	Leads with no activity	Only leads without next activity appear.
Overdue activity	Leads with overdue activity	Only overdue leads appear.
Combined filters	Owner + label + city + activity	AND logic works or documented behavior confirmed.

16. Deals Module Test Design

Deals verifies the pipeline lifecycle. It is high risk because state changes must persist across board movement, won/lost status, lost reason, archive, and reload.

16.1 Deals Test Cases

ID	Scenario	Agent action	Expected result
DEAL-001	Open Deals	Open Deals from sidebar.	Pipeline board loads.
DEAL-002	Blueprint controls	Verify Add Deal, New Pipeline, stage columns, Forecast if available.	Expected controls visible.
DEAL-003	Create deal	Add deal with name, pipeline, stage, value, product, close date, probability.	Deal card appears.
DEAL-004	Pipeline auto-select	Convert from lead context.	Pipeline and stage auto-selected as expected.
DEAL-005	Move stage	Drag or move card from Qualified to Demo Scheduled/other stage.	Card moves and persists after reload.
DEAL-006	Mark won	Mark deal as Won.	Deal status becomes Won and persists.
DEAL-007	Mark lost validation	Attempt lost without reason.	Lost reason required.
DEAL-008	Mark lost valid	Select lost reason and save.	Deal status becomes Lost with reason saved.
DEAL-009	Archive deal	Archive staging deal.	Deal moves to Archive and becomes read-only.
DEAL-010	Unarchive deal	Unarchive deal.	Deal returns to active pipeline.
DEAL-011	Delete full regression	Delete allowed staging deal in destructive mode.	Deal removed and absent after reload/search.
DEAL-012	Forecast	Check forecast numbers.	Forecast reflects visible deal values according to product rules.

16.2 Deal Stages and Lost Reasons

Area	Expected values from blueprint
Stages	Qualified, Demo Scheduled, Demo Completed, Proposal Made, Payment Followup, Won/Lost
Lost reasons	Price is too high; Lost to competitor; Missing required features; Not Interested; No Requirement; Need old in future; No Response; Junk Lead; No proper follow-up
Fields	Deal Name, Pipeline, Pipeline Stage, Deal Value, Product, Expected Close Date, Probability, Weighted Deal Value

17. Activities Module Test Design

ID	Scenario	Agent action	Expected result
ACT-001	Open Activities	Open Activities page.	Activity page loads.
ACT-002	Schedule from lead	Create activity from lead detail.	Activity appears in lead and Activities module.
ACT-003	Activity due states	Create today/upcoming/overdue examples if possible.	Correct state appears.
ACT-004	Complete task	Mark task complete.	Completion state and history update.
ACT-005	Outcome	Select Interested/Left Message/No Response/Not Interested/Not able to reach.	Outcome persists.
ACT-006	Refresh	Reload Activities after schedule/complete.	State persists.

18. Products and Subscription Test Design

Products includes product creation/editing and subscription plan behavior. These tests should run after core Leads/Deals because products can feed Deal values and conversion flows.

ID	Scenario	Agent action	Expected result
PROD-001	Open Products	Open Products from sidebar.	Products page loads.
PROD-002	Blueprint controls	Verify Add/Edit Products, Import, Export, Manage Columns.	Expected controls visible.
PROD-003	Add product	Create QA product with price/tax.	Product appears in table and persists.
PROD-004	Subscription plan	Create Growth 1 Year / 2 Year plan if allowed.	Plan values save correctly.
PROD-005	Billing frequency	Validate One time, Weekly, Monthly, Quarterly, Every 6 months, Annually.	Dropdown/options match product rule.
PROD-006	Mark inactive	Mark staging product inactive.	Status changes and persists.
PROD-007	Edit product	Change name/price/tax/frequency.	Updated values persist after reload.
PROD-008	Delete destructive	Delete staging product if allowed and not linked.	Correct confirmation and final removal.

19. Settings, Reports, Campaigns, Referrals, Tasks

These modules are visible in the CRM navigation blueprint, but their detailed screens are not fully specified in the uploaded file. QaAgent-Pro should handle them in two levels: smoke/conformance now, deeper workflows after the product team provides expected rules.

Module	Initial agent coverage	Next information needed
Settings	Page opens, role controls visible, destructive changes blocked.	Exact settings screens and allowed test modifications.
Reports	Page opens, filters/download controls visible, no console/network failures.	Report types, expected numbers, export rules.
Campaigns	Page opens and primary controls visible.	Campaign creation/send safety rules.
Referrals	Page opens and table/action controls visible.	Referral creation/source rules.
Tasks	Page opens, task list states visible.	Task create/complete/assign rules.

20. Import, Export, and Manage Columns

Area	Agent coverage	Notes
Import	Verify import dialog, template download, invalid file validation, safe dry-run if supported.	Do not upload real customer data.
Export	Verify export action/menu and file download in staging.	Sensitive export should be blocked unless test tenant.
Manage Columns	Hide/show columns, verify table changes, reload persistence.	Column preference behavior must be product-confirmed.
Bulk actions	Bulk archive/unarchive/convert/delete if allowed.	Must use staging and confirmation evidence.

21. UX and Product Improvement Engine

The user explicitly needs the agent to say what to improve and what to define for the next build. This is separate from defect detection.

21.1 UX Review Checklist

UX dimension	What agent checks
Discoverability	Can primary actions be found without hidden hover-only controls?
Feedback	Does save/archive/delete/convert show clear success or error state?
Recovery	Can user undo/unarchive/retry after failure?
Empty states	Do empty lists and search no-results explain what to do next?
Validation copy	Are required/invalid/duplicate messages clear and near the field?
Loading states	Are saves/searches/filters visibly loading and not double-submittable?
Keyboard/focus	Can forms, modals, drawers, and table actions be used with keyboard basics?
Mobile/laptop layout	Are tables/actions usable at common business laptop sizes?
Information hierarchy	Are owner, source, value, next activity, status easy to scan?

21.2 Next Build Recommendation Rules

Trigger	Backlog recommendation example
Action exists but is hidden or unclear.	Add visible row action menu with accessible labels.
Toast appears but table does not update.	Add optimistic update rollback or automatic table refresh after save.
Refresh loses data.	Fix persistence/API save logic and add backend verification.
Repeated filter confusion.	Add active filter chips and clear-all behavior.
Archived records editable or unclear.	Enforce read-only UI with unarchive CTA.
Bulk action risky.	Add selected-count confirmation and preview list.
No feature from blueprint.	Add missing control or update blueprint with product decision.

21.3 Output Example

Next Build Recommendation

ID: QAP-NB-014

Type: UX Improvement + Data Reliability

Module: Leads Table

Recommendation: Add explicit table Refresh action and automatically refresh table after archive/unarchive.

Reason: Agent found records remained visible after archive until browser reload.

Priority: P1

Acceptance Criteria:

1. After archive, record disappears from Inbox within 2 seconds.
2. Archive tab shows the same record.
3. Browser reload preserves the state.
4. Excel retest status becomes PASS.

22. Implementation Roadmap

Build in phases. Do not try to complete all CRM modules first. The first working demo should prove one module deeply: Leads with refresh/persistence and Excel report.

Phase	Scope	Deliverable
0	Research and blueprint conversion	Excalidraw-to-blueprint YAML, v2 PRD/TDD, test matrix.
1	Repo scaffold	New qaagent-pro repo, TypeScript, Playwright, env, CLI, quality gate.
2	Browser engine	Login/session manager, screenshots, trace, console/network capture, state snapshot.
3	Report engine	Excel workbook with Bug Report, UX Issues, Feature Gaps, Next Build, Refresh sheets.
4	Leads MVP	Leads create/validate/search/detail/activity/convert/ archive/unarchive/refresh suite.
5	Deals	Pipeline, create, move, won/lost, archive, delete destructive.
6	Activities and Products	Schedule/complete/outcome; product/subscription plan tests.
7	UX and blueprint audit	Missing controls, UX issues, accessibility checks, next-build backlog.
8	Full regression	All modules, destructive staging suite, retest workflow, CI-ready run.

22.1 First 7-Day Build Plan

Day	Focus	Done when
1	Repo scaffold and env contract	npm install, playwright install, quality gate skeleton works.
2	Login/session/evidence	Agent logs in and captures screenshot/console/network logs.
3	Leads page object	Open Leads, verify blueprint controls, search/table checks.
4	Add Lead and validation	Required, invalid email/mobile, valid create, duplicate behavior.
5	Refresh suite	Create/edit/archive/unarchive persistence after reload.
6	Excel report	Bug Report, UX Issues, Feature Gaps, Refresh, Summary sheets with screenshots.
7	Demo run and fix pass	npm run qa:leads creates final Excel report without manual steps.

22.2 First MVP Acceptance

- npm run qa:leads works from a clean checkout after env setup.
- Agent logs in and opens Leads.
- At least 20 Leads scenarios are implemented.
- Refresh/persistence tests are included.
- Excel report includes embedded screenshots for failed findings.
- Bug, UX, Feature Gap, Refresh, and Next Build sheets are present.
- No production or real-message action can run accidentally.
- Destructive tests are excluded from default MVP run.

23. CLI, Configuration, and Run Manifest

23.1 Commands

```
npm run qa:blueprint
npm run qa:smoke
npm run qa:leads
npm run qa:deals
npm run qa:activities
npm run qa:products
npm run qa:refresh
npm run qa:ux
npm run qa:regression
npm run qa:full-destructive
npm run qa:retest -- --bug QAP-BUG-001
npm run quality:gate
```

23.2 Run Manifest

```
{
  "runId": "QAP-2026-06-19-001",
  "product": "Zybra CRM",
  "environment": "staging",
  "mode": "leads-mvp",
  "blueprintVersion": "excalidraw-2026-06-19",
  "safetyProfile": "staging-write",
  "allowArchive": true,
  "allowDelete": false,
  "headless": false,
  "report": {
    "format": "xlsx",
    "embedScreenshots": true
  }
}
```

24. Selector and Testability Requirements for Developers

To make the agent stable, Zybra CRM should add data-testid attributes to important controls. This is not required for a prototype but is strongly recommended before release-gate automation.

Element	Recommended data-testid
Sidebar Leads	nav-leads
Sidebar Deals	nav-deals
Leads Add button	leads-add-button
Lead save button	lead-save-button
Lead form contact name	lead-contact-name-input
Lead form mobile	lead-mobile-input
Lead form email	lead-email-input
Leads search	leads-search-input
Lead row action menu	lead-row-action-menu
Lead archive action	lead-archive-action
Lead unarchive action	lead-unarchive-action
Lead convert action	lead-convert-action
Deals board	deals-pipeline-board
Deal stage column	deal-stage-column
Product add button	products-add-button
Table refresh button	table-refresh-button

Refresh testability request

If the CRM has no visible refresh control, the agent will still test browser reload and navigation refresh. But a visible table refresh action should be added if the product expects users to manually refresh list data.

25. Quality Gates and CI

Gate	Checks
Typecheck	TypeScript compiles with no errors.
Smoke test	Login and sidebar/module navigation pass.
Report sanity	Excel workbook generated, contains expected sheets, screenshots embedded.
Secret scan	No credentials/auth state committed.
Safety scan	Destructive commands blocked unless flags and staging URL pass.
Flaky check	Retries and quarantines documented; no hidden failures.
Accessibility baseline	Automated scan results recorded for critical pages.
Performance baseline	Page/search/save/archive/delete timing captured.

26. Risks and Mitigations

Risk	Impact	Mitigation
Changing CRM UI	Selectors fail.	Use data-testid, role/text locators, selector registry, controlled healing.
Agent over-autonomy	False bugs or destructive behavior.	Deterministic playbooks and safety gates.
No expected rules documented	Ambiguous bug reports.	Blueprint YAML with confidence levels and product confirmation items.
Toast/API mismatch	False pass.	Always verify final UI state, network response, and refresh persistence.
Production data damage	Severe.	Staging allowlist, destructive env flags, audit log, QA_AGENT prefix.
Import/export sensitive data	Privacy risk.	Test tenant only; export disabled by default.
PDF/report unreadability	Stakeholder friction.	Excel report layout rules and visual QA before delivery.
Browser-agent unreliability	Inconsistent results.	Fixed playbooks and explicit validators.

27. Open Questions for Zybra Team

These are the only questions that should block final implementation. The agent can start with assumptions, but these answers will improve accuracy.

1. What is the staging CRM URL and test tenant name?
2. What role should the QA login use: admin, owner, sales user, or limited user?
3. Which fields are officially required for Lead and Deal creation?
4. Should duplicate email/mobile be blocked or warned but allowed?
5. Should filters persist after browser refresh or reset intentionally?
6. Should Manage Columns persist per user or reset per session?
7. Can the agent delete only QA_AGENT_ data, or any staging data selected in the destructive suite?
8. Are Email/WhatsApp/Call actions to be opened only, or sent in a sandbox?
9. Which reports/campaigns/referrals/tasks flows are MVP versus future?
10. Can developers add data-testid attributes to stabilize automation?

28. Final Recommendation

Final recommendation

Start with QaAgent-Pro as a new standalone TypeScript + Playwright repo. Implement Leads MVP first with refresh/persistence and Excel reporting. Then add Deals, Activities, Products, UX audit, feature-gap detection, and destructive

full regression. This matches the Zybra CRM blueprint while also producing next-build recommendations instead of only bug reports.

29. Research References

Source	Used for design decision
BAKUGOS1/QaAgent GitHub repository	Existing local-first TypeScript + Playwright QA-agent concepts, Excel reports, safety model, selector healing.
QaAgent AGENTS.md	QA standards, report style, safety rules, profiles, memory rules, done criteria.
Playwright Authentication documentation	Auth storageState pattern and auth-file sensitivity.
Playwright Best Practices	User-visible behavior, isolation, maintainability guidance.
Playwright Test Generator documentation	Generating locators and preserving authenticated state.
Playwright Page Object Models documentation	Maintainable page-object structure for large suites.
Playwright Trace Viewer, Screenshots, Network, Console APIs	Evidence capture model.
Playwright Accessibility Testing documentation	Automated accessibility plus manual assessment limitation.
WAREX browser-agent reliability research	Real-world instability causes agent success-rate drops.
WebSuite web-agent failure research	Browser agents need action-level failure diagnosis.
Trustworthy GUI Agents survey	Autonomous GUI agents need safety, transparency, and reliability controls.